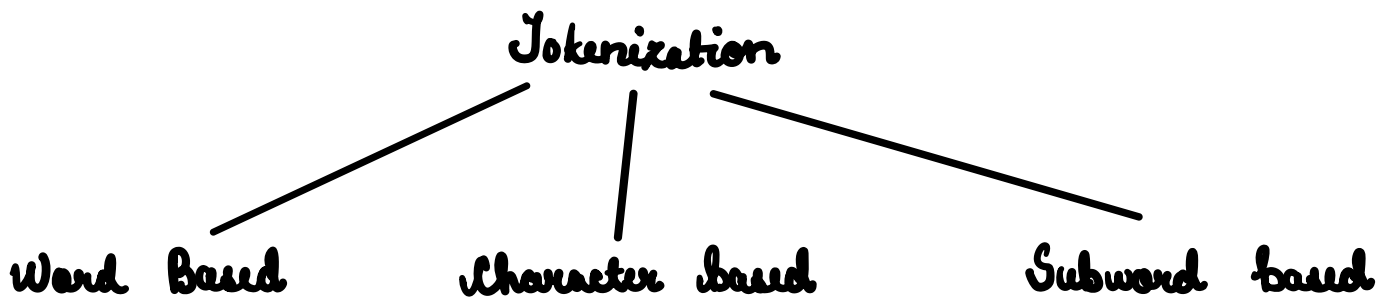


Tokenization :

breaking raw text into smaller units.



WordPiece Tokenization : (Used by BERT, FinBERT)

Instead of splitting only by words, it breaks text into frequent subword units.

" Fed signals hawkish stance amid inflation "



[" fed ", " signals ", " hawk ", " # ish ", " stance ", " amid ", " inflation "]



continuation of
a word



One-hot encoding

Token IDs



Embedding lookups

↓ mapping each token ID to a dense vector in embedding space

Dense Vectors



Transformers layer

Attention :

Mechanism that assigns importance weights to different tokens based on relevance to a query

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Query

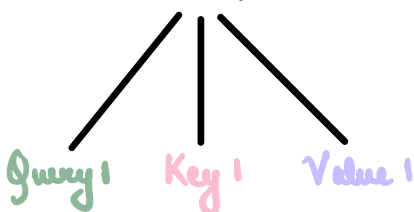
used to ask for information

Key

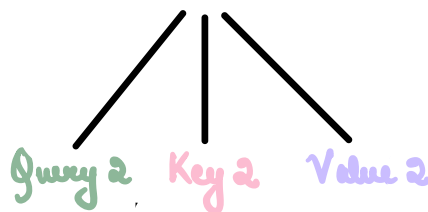
provides that information

Value

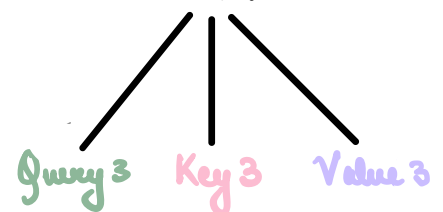
Token 1



Token 2



Token 3



Here Q and K have dimensions d_k

Similarity score:

Measure of relevance between a query vector and a key vector.

$$S_{ij} = Q_i K_j^T$$

since QK^T is a dot product, it grows with dimension

Softmax converts scores to probabilities

It struggles when the data has:

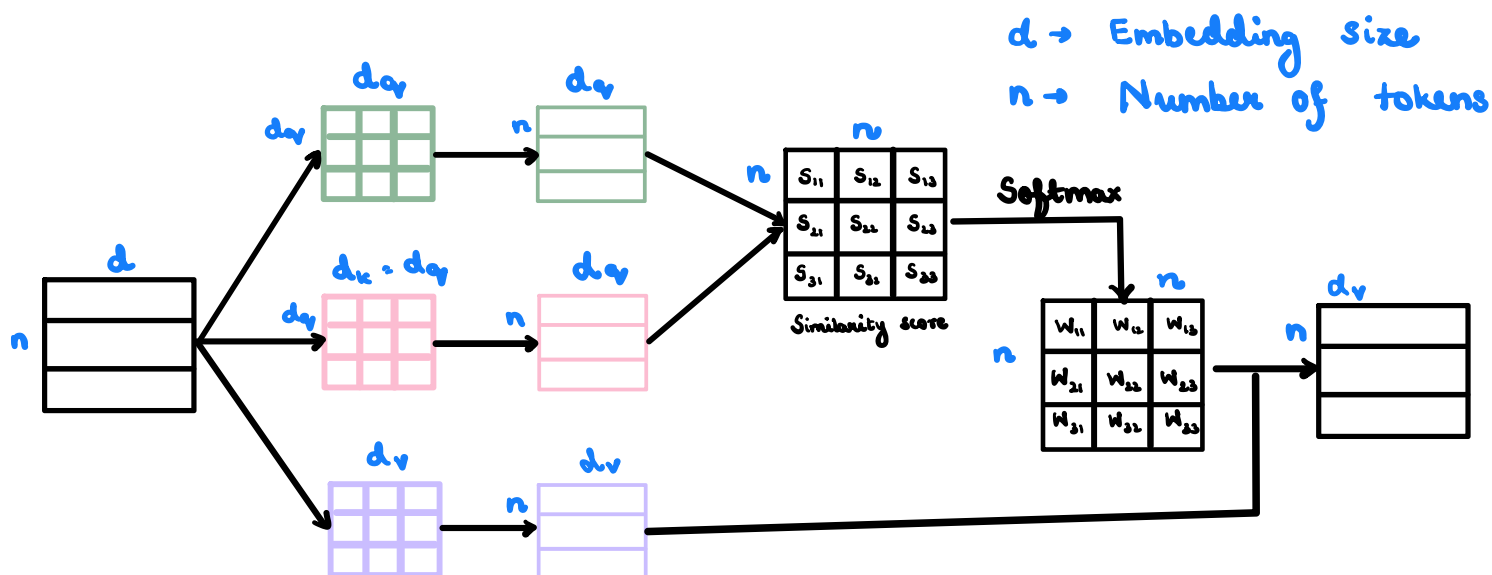
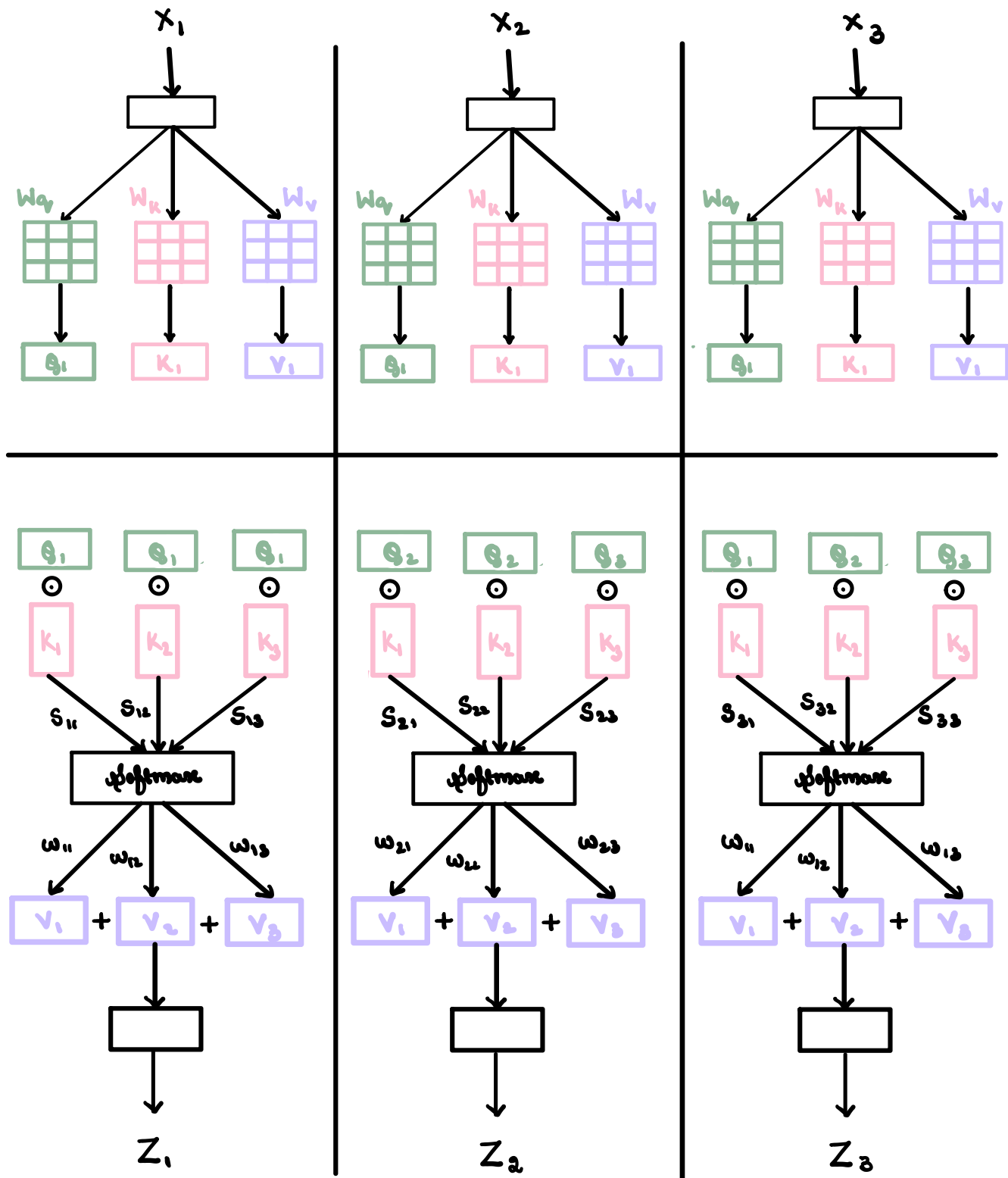
- large magnitude
- high variance

Attention Weights:

Weights assigned to each token after applying softmax to the similarity score.

$$\text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right)$$

→ dividing by $\sqrt{d_k}$
normalizes the score
to avoid vanishing gradient



Causal Attention :

each position can only attend to itself and the past, never the future

	T_1	T_2	T_3	T_4
T_1				
T_2				
T_3				
T_4				

Prevent tokens from attending future tokens

	T_1	T_2	T_3	T_4
T_1				
T_2				
T_3				
T_4				

+

Masked Matrix

0	$-\infty$	$-\infty$	$-\infty$
0	0	$-\infty$	$-\infty$
0	0	0	$-\infty$
0	0	0	0

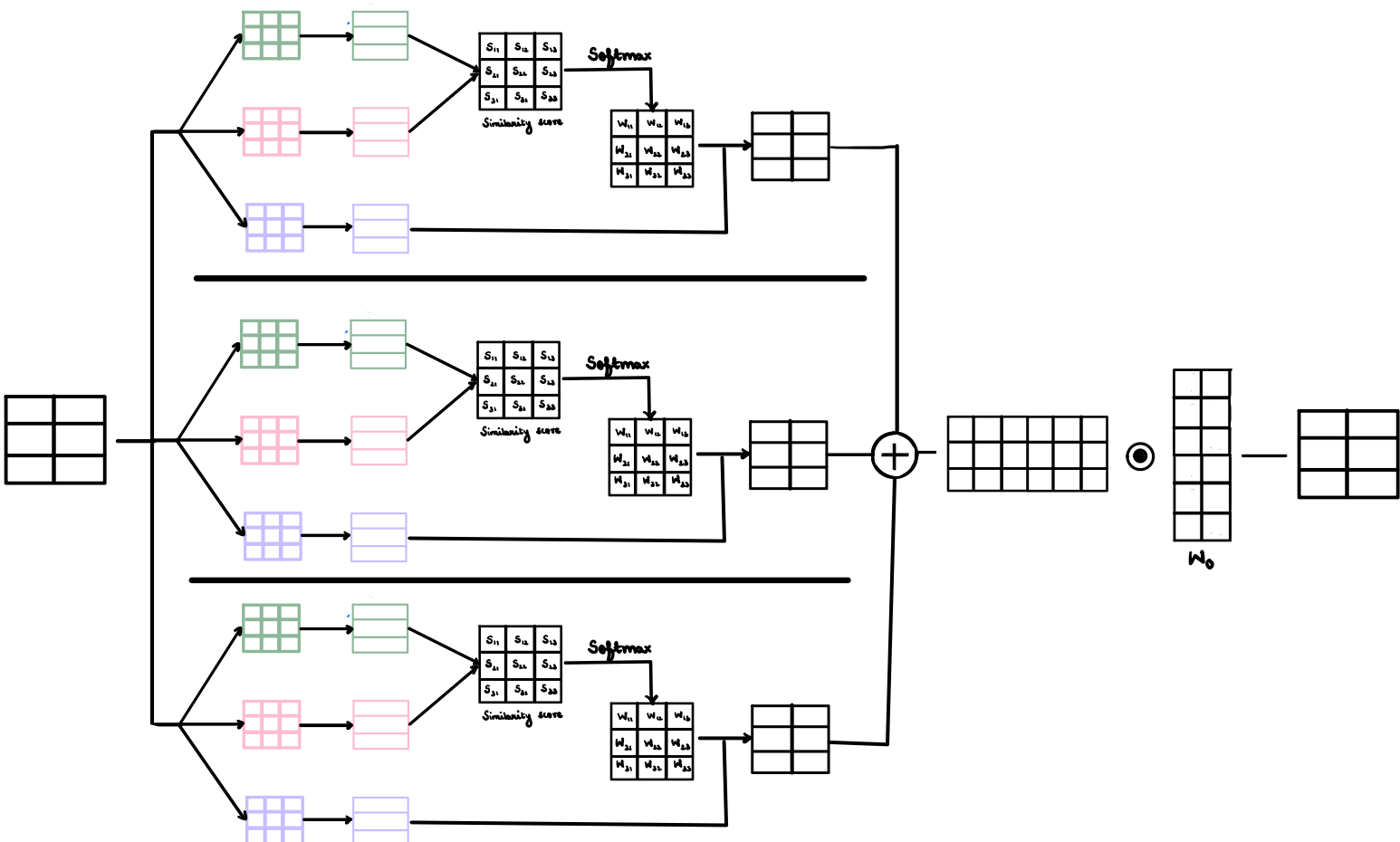
We use $-\infty$ to mask because upon applying softmax function value becomes zero.

Multihead Attention :

Multiple self attention mechanisms are used in parallel by breaking the embeddings into smaller subspaces.

⇒ allows model to understand different perspectives of the input simultaneously.

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \text{head}_2, \dots, \text{head}_n) W^O$$



Take multiple mechanism heads
each with its own learnable weights



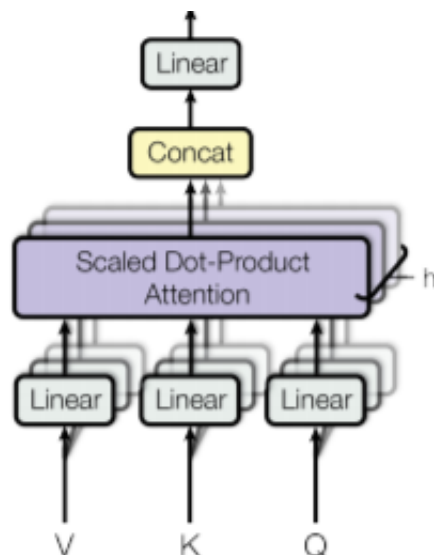
Each head produces an output
embedding in a lower dimension

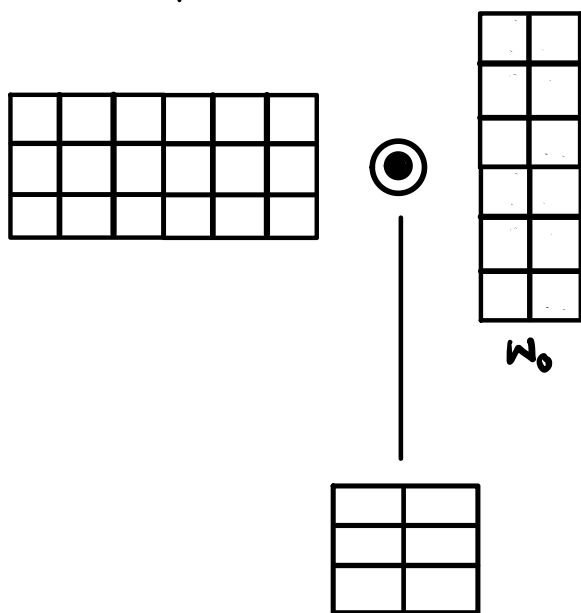
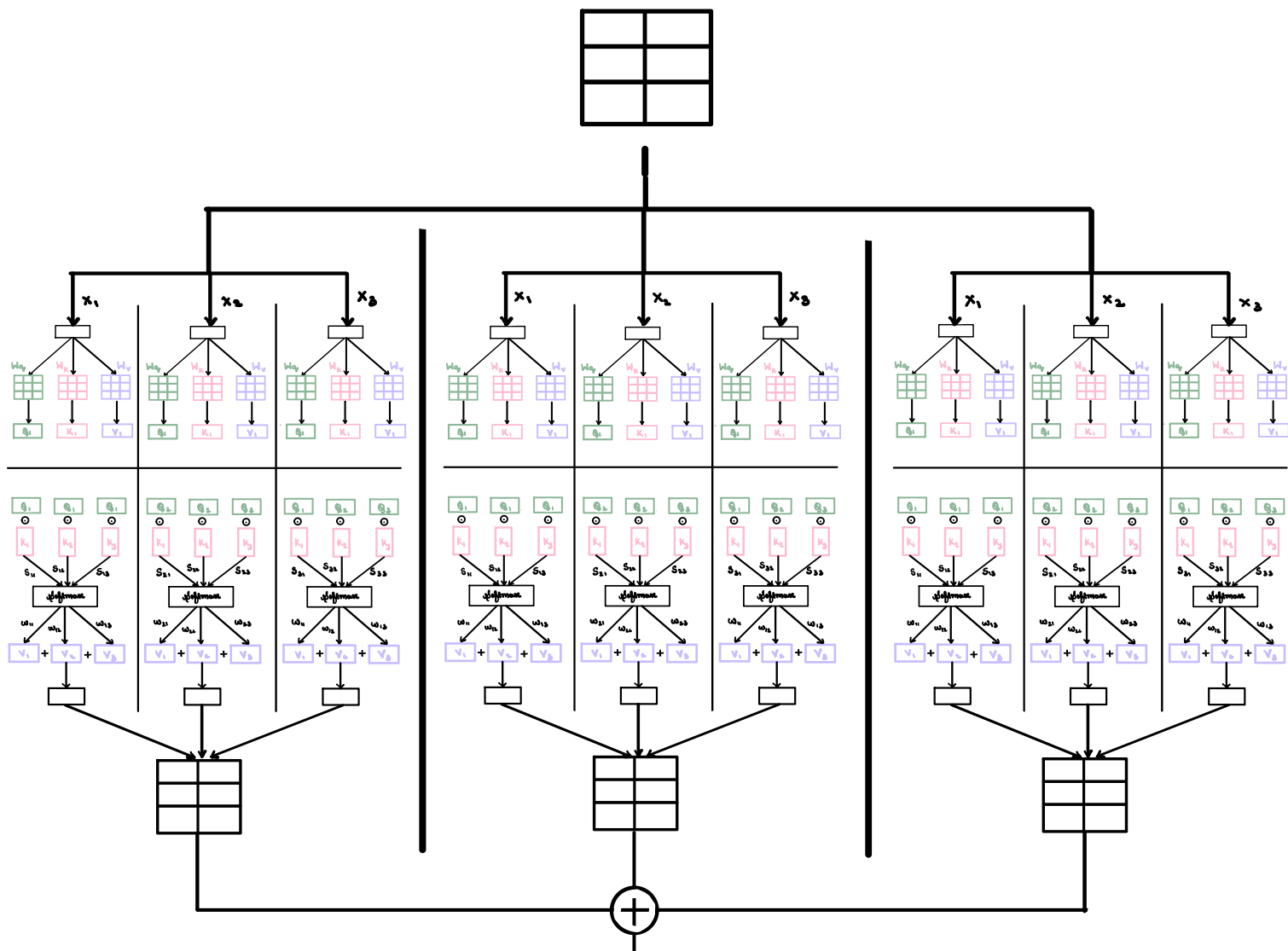


Outputs are concatenated



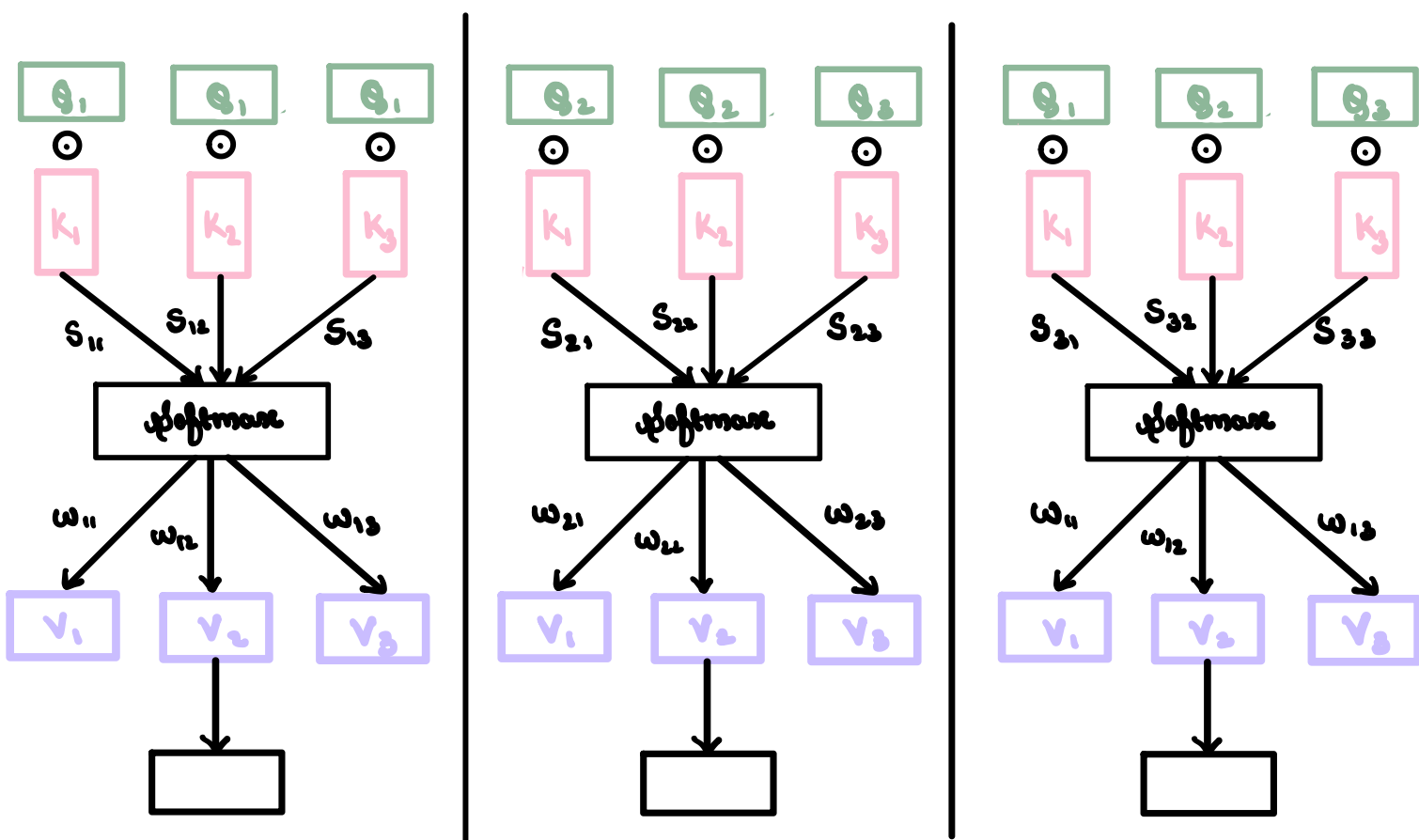
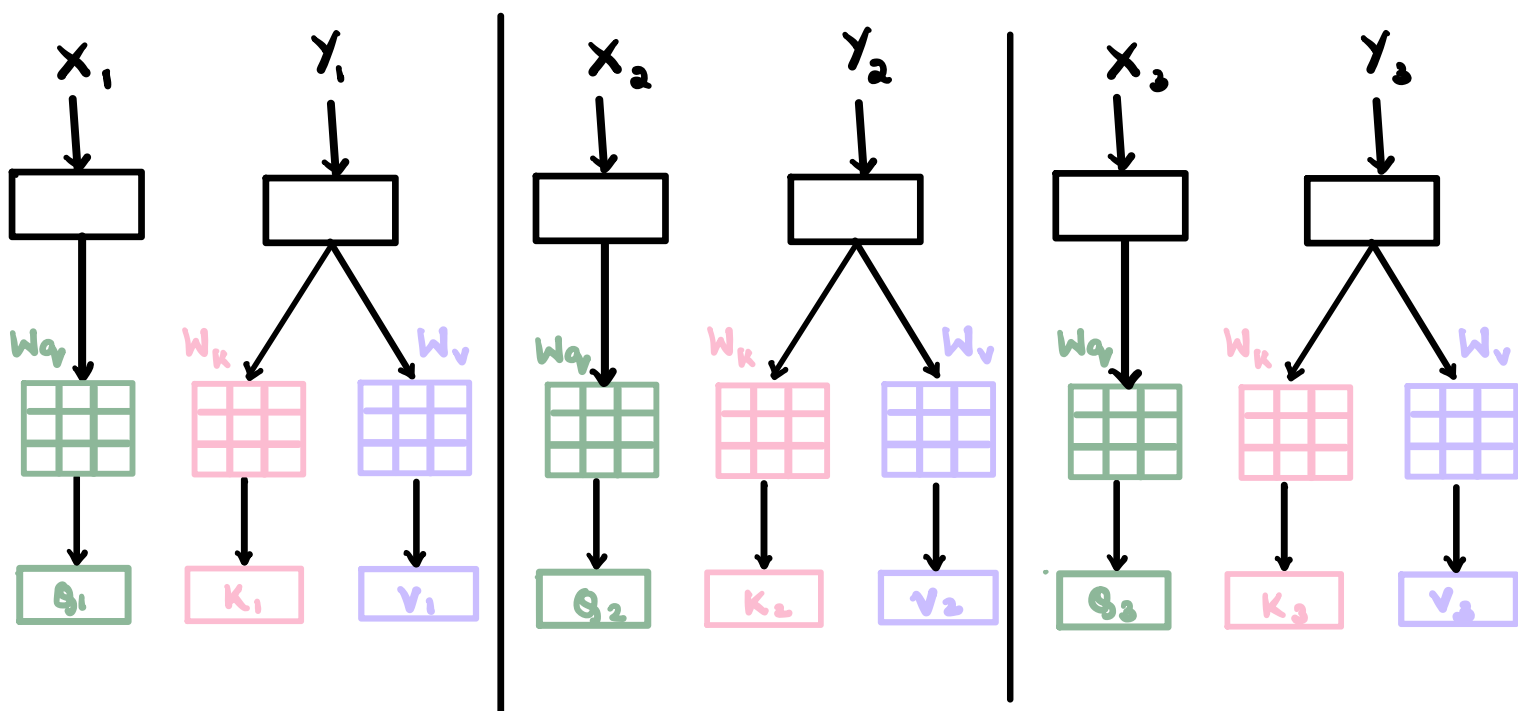
linearly projected to get final contextual
representation (Allows the model to combine
information from multiple attention heads
capturing different types of dependencies
between words)





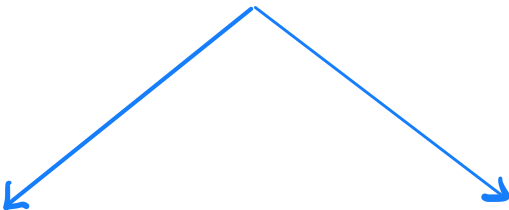
Cross Attention :

Attention mechanism where query comes from one sequence, while the Key and Value come from another sequence



Component 1: Dynamic News Graph Builder

D_1	P_1	News 1
D_2	P_2	No News
D_3	P_3	News 3



D_1	P_1
D_2	P_2
D_3	P_3

D_1	News 1
D_2	No News
D_3	News 3

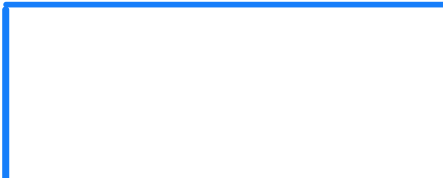
Finbert Embedding

	768			
D_1				
D_2				
D_3				

PCA dimensionality reduction

	16			
D_1				
D_2				
D_3				

E (Embedding Matrix)



0	1	2
1	0	1
2	1	0

Distance Matrix

$$\text{dist}_{ij} = |i - j|$$

Exponential Decay

gamma $\gamma \rightarrow 0.1$
Decay rate

Temporal Prior Matrix

$$A_{ij}^{\text{tmp}} = \exp(-\gamma |i - j|)$$

cosine similarity

$$\text{Cos. Sim}(A, B) = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

	D_1	D_2	D_3
D_1	S_{11}	S_{21}	S_{31}
D_2	S_{12}	S_{22}	S_{32}
D_3	S_{13}	S_{23}	S_{33}

Similarity score

1	0	1
0	0	0
1	0	1

$$m = \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} \quad 0 \Rightarrow \text{no news}$$

News Availability
Mask Matrix

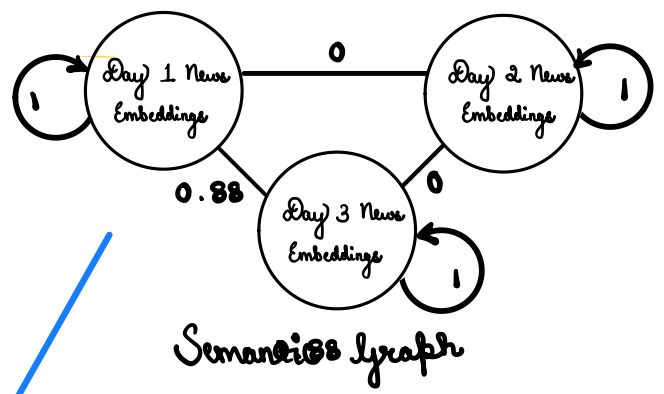
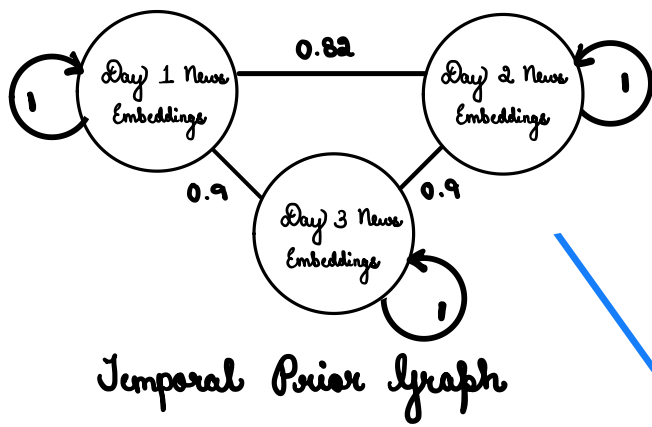
$$M = mm^T$$

Semantic Matrix

$$A_{ij}^{\text{sem}} = m_i m_j \sim (10(S_{ij} - \tau))$$

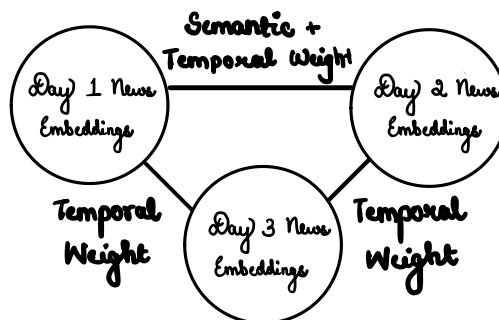
$S_{ij} - \tau$ is tiny
Feeding this into a standard sigmoid function will hover stubbornly around 0.5
So, we use scaled sigmoid function instead of standard sigmoid function

τ is the learnable threshold



Single Adjacency Matrix

$$A_{ij} = \alpha_g A_{ij}^{\text{sem}} + \beta_g A_{ij}^{\text{tmp}}$$



Zero the Diagonal
Subtract identity matrix
to remove self loops

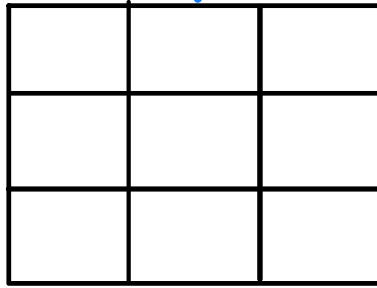
Symmetric Normalized Laplacian Matrix

$$\hat{L} = \frac{A}{\sqrt{D_{ii} D_{jj}}}$$

Random Walk Normalization $D^{-1}A$
(solves gradient vanishing or
gradient exploding)

Guaranteed Real Eigenvalues $D^{-1}A$
Matrix is only guaranteed to have real
eigen values and orthogonal eigenvectors
if its symmetric. $D_{ii} \neq D_{jj}$
 $D^{-1}A$ is asymmetric. By applying the
inverse square root to both sides, the
model scales the edges symmetrically

By leveraging this specific formula, the convolution avoids $O(N^4)$ computation steps for eigen decomposition entirely.



Chebyshev Spectral Convolution

$$Z = \sum_{k=0}^K X_k W_k + b$$

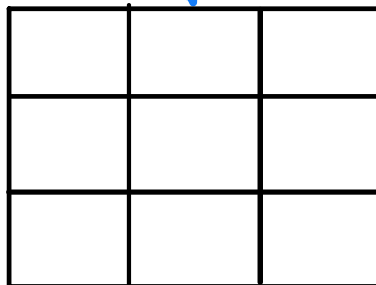
where:

- Z is the final graph-aware output matrix
- K is the order of Chebyshev polynomial
(defines information diffusion in how many steps)
- X_k represents the news features after k hops.
- W_k is the learnable weight matrix specific to k hops
- b is the learnable bias vector.

Every node updates its state by pulling in a fraction of its neighbor's states.

Hop 1: Node A blends with Node B

Hop 2: Node A blends more with Node C and so on $\rightarrow$ Vanishing gradients

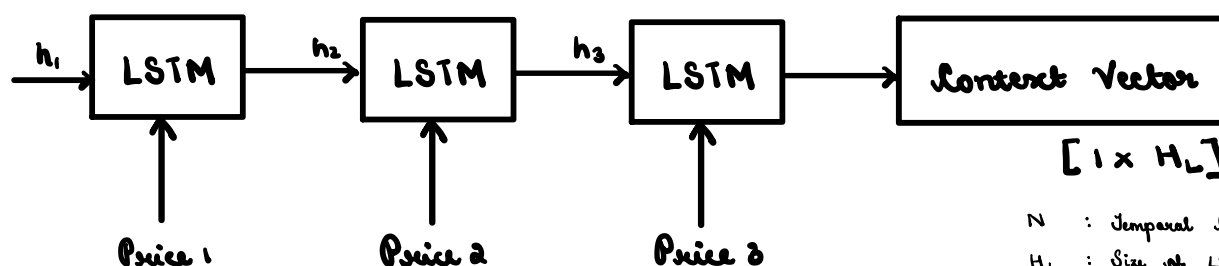


Skip connection

$$Z_{\text{final}} = Z + E$$

$$Z_{\text{out}} = \text{LayerNorm}(\text{GELU}(Z_{\text{final}}))$$

Component 2: LSTM Encoder



- N : Temporal lookback window
- H_L : Size of LSTM hidden state
- H_G : Feature size of news embedding
- H_{Att} : Shared hidden dimension you configure for the attention layer

Component 3 : Cross Attention Layer

